

1 Fall Detection Using 3D Convolutional Neural Network

1.1 Overview

This project implements a 3D Convolutional Neural Network (3D CNN) to automatically detect human falls from video data. Fall detection is important in healthcare applications such as elderly care and hospital monitoring, where timely detection can help prevent serious injuries. The model classifies short video clips into two categories: fall and non-fall (normal daily activities).

1.2 Objective

The objective of this project is to design and evaluate a deep learning model that can learn spatial and temporal patterns from video sequences and accurately distinguish between fall and non-fall activities.

1.3 Dataset

The UR Fall Detection Dataset (URFD) is used in this project.

Dataset characteristics:

- Total videos: approximately 70
- Classes: Fall and Non-Fall (Activities of Daily Living)
- Resolution: 320×240 pixels
- Recorded using Kinect camera
- Application: fall detection for healthcare monitoring

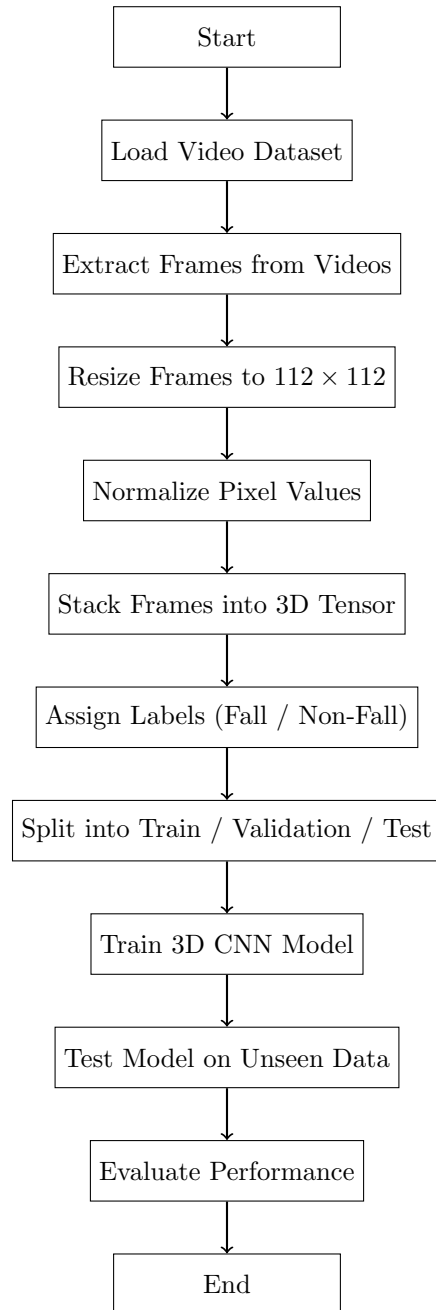
Dataset source (Kaggle mirror): <https://www.kaggle.com/datasets/shahliza27/ur-fall-detection-dataset>

1.4 Workflow

The complete processing pipeline is as follows:

- Load video dataset
- Extract frames from each video
- Resize and normalize frames
- Stack frames to form 3D tensors
- Train a 3D CNN model
- Predict fall or non-fall
- Evaluate model performance

1.5 Flowchart



1.6 Data Preprocessing

Each video is processed by extracting a fixed number of frames (for example, 16 frames). This ensures that all video inputs have the same temporal length. Each frame is resized to 112×112 pixels and normalized by dividing pixel values by 255 so that they lie in the range $[0, 1]$. The frames are stacked sequentially to form a 3D tensor of shape $(frames, height, width, channels)$, which is used as the input to the 3D CNN model.

1.7 Model Architecture

A 3D Convolutional Neural Network is used to capture both spatial and temporal information from video data. The model consists of 3D convolution layers to extract motion and posture features, batch normalization layers to stabilize training, max pooling layers to reduce feature dimensions, a fully connected (dense) layer for classification, and a sigmoid output layer for binary prediction (fall or non-fall). This architecture allows the model to learn how body posture changes over time and use this information to detect falls.

1.8 Training Process

The dataset is divided into 70% training data, 15% validation data, and 15% testing data. The model is trained using the Adam optimizer and binary cross-entropy loss function. During training, predictions are generated, errors are computed, and model weights are updated using backpropagation over multiple epochs.

1.9 Evaluation Metrics

The trained model is evaluated using accuracy, precision, recall, F1-score, and Area Under the ROC Curve (AUROC). A confusion matrix is also used to analyze correct and incorrect predictions. Recall is especially important because missing a fall event can be dangerous in healthcare applications.

1.10 Error Analysis

Some incorrect predictions occur due to camera angle variations, occlusion of the subject, motion blur, and slow or unclear fall movements. These factors reduce the clarity of motion patterns and make classification more difficult in certain cases.

1.11 Conclusion

This project demonstrates the application of a 3D Convolutional Neural Network for fall detection using video data. By learning spatial and temporal features together, the model can effectively classify fall and non-fall activities. The

results show the potential of deep learning methods for healthcare monitoring and safety systems.